

V O R T E X T E C H

Cyber Security Internship 2026

Week 3 Security Audit

Basic Security Audit of a Sample Website

Target: OWASP Juice Shop • **URL:** http://localhost:3000
Tools: Browser DevTools (manual testing)
Report date: 17 September 2026



REPORT SCOPE

Based on manual, browser-based testing (DevTools + authentication testing). Three findings are documented across three vulnerability categories: SQL Injection, Sensitive Data Exposure, and Cross-Site Scripting.

Prepared as part of the Vortex Tech Cyber Security Internship 2026 — Week 3 of 4 (Intermediate)

1. EXECUTIVE SUMMARY

A security audit was performed against a locally hosted OWASP Juice Shop instance using manual, browser-based testing (DevTools + authentication testing). The testing produced three findings:

- SQL Injection — Admin authentication bypass
- Sensitive Data Exposure — Internal fields exposed in API response
- Cross-Site Scripting — Input accepted without sanitization

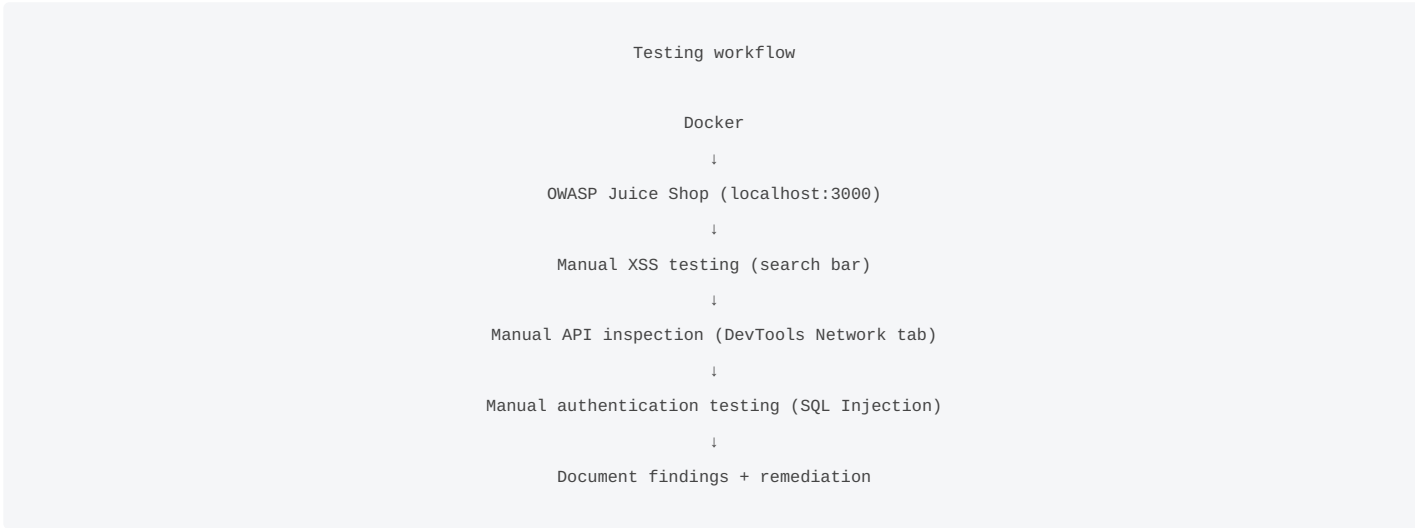
The audit therefore documents three findings across three distinct vulnerability categories:

- SQL Injection (Critical)
- Sensitive Data Exposure / Information Disclosure (Medium)
- Cross-Site Scripting — Input accepted (Medium)

2. METHODOLOGY

The assessment followed the Week 3 task requirements:

- Use a legal practice target — OWASP Juice Shop via Docker.
- Perform manual testing using browser DevTools.
- Document each finding: title, discovery steps, category, impact, remediation.



Finding 1 — SQL Injection: Authentication Bypass

CRITICAL

Category	Injection — OWASP Top 10 2021 A03
CWE	CWE-89
Affected endpoint	POST http://localhost:3000/rest/user/login
Discovery method	Manual testing

WHAT WAS FOUND

The login form is vulnerable to SQL Injection. By entering a crafted payload in the email field, it was possible to bypass authentication entirely and log in as the administrator account (admin@juice-sh.op) without knowing the password.

STEPS TAKEN TO FIND IT

1. Navigated to http://localhost:3000/#/login.
2. Entered the following SQL Injection payload in the Email field: ' OR 1=1--
3. Entered any value in the Password field: anything
4. Clicked Log in.
5. The application authenticated successfully as admin@juice-sh.op.

EVIDENCE SCREENSHOTS

- screenshots/06-sqli-payload-entered.png — payload entered in login form
- screenshots/07-sqli-admin-menu.png — admin@juice-sh.op shown in account menu
- screenshots/08-sqli-admin-home.png — full admin session on home page

POTENTIAL REAL-WORLD IMPACT

SQL Injection in an authentication endpoint is critical severity. An attacker can:

- Bypass login without valid credentials
- Access administrator accounts and all admin functionality
- Potentially extract, modify, or delete the entire database
- Escalate to full application compromise

PRACTICAL REMEDIATION

Use parameterized queries (prepared statements) for all database operations. Never concatenate user input directly into SQL strings.

```
// Vulnerable
db.query("SELECT * FROM users WHERE email='" + email + "'");

// Secure
db.query("SELECT * FROM users WHERE email = ?", [email]);
```

Category	Sensitive Data Exposure / Information Disclosure — OWASP Top 10 2021 A02
CWE	CWE-200
Affected endpoint	GET http://localhost:3000/rest/products/search?q=
Discovery method	Manual testing — Browser DevTools Network tab

WHAT WAS FOUND

The product search API returns internal database fields in plain text in its JSON response that should not be exposed to the client. Fields including `deluxePrice` , `createdAt` , `updatedAt` , and `deletedAt` are visible to any user in the browser.

STEPS TAKEN TO FIND IT

1. Opened browser DevTools (F12) → Network tab.
2. Entered `<script>alert(1)</script>` in the Juice Shop search bar.
3. Clicked the `search?q=` request in the Network tab.
4. Opened the Response tab.
5. Observed the full JSON response containing internal fields for every product:

```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "price": 1.99,
      "deluxePrice": 0.99,
      "createdAt": "2026-09-16 20:25:58.656 +00:00",
      "updatedAt": "2026-09-16 20:25:58.656 +00:00",
      "deletedAt": null
    }
  ]
}
```

EVIDENCE SCREENSHOTS

- `screenshots/02-xss-network-request.png` — Network tab showing `search?q=` request
- `screenshots/03-api-sensitive-data-1.png` — Response tab showing exposed fields (top)
- `screenshots/04-api-sensitive-data-2.png` — Response tab showing exposed fields (scrolled)

POTENTIAL REAL-WORLD IMPACT

Exposing internal fields such as `deluxePrice`, `deletedAt`, and database timestamps leaks:

- Internal pricing tiers that give attackers business intelligence
- Database schema information useful for further attacks
- Soft-delete status of records that should be invisible to users

PRACTICAL REMEDIATION

API responses should only include fields the client legitimately needs. Use an explicit allowlist of fields to serialize, rather than returning the full database model object.

Category	Cross-Site Scripting — OWASP Top 10 2021 A03
CWE	CWE-79
Affected endpoint	GET http://localhost:3000/#/search?q=
Discovery method	Manual testing

WHAT WAS FOUND

The search bar accepts and processes HTML/JavaScript tags in user input. When `<script>alert(1)</script>` was entered, the payload was passed to the server encoded in the URL (%2Fscript) and the API returned product data without blocking the request. No alert popup fired, indicating the front-end renders results safely in this case — however the server does not sanitize or reject the input, which is a vulnerability.

STEPS TAKEN TO FIND IT

1. Navigated to http://localhost:3000.
2. Clicked the search icon and entered: `<script>alert(1)</script>`
3. Pressed Enter.
4. Observed the URL changed to: `localhost:3000/#/search?q=<script>alert(1)<%2Fscript>`
5. No alert popup appeared — the front-end encodes the output.
6. Checked DevTools Network tab — `search?q=` returned HTTP 304 and the API still processed the request and returned product data.

EVIDENCE SCREENSHOTS

- screenshots/01-xss-search-payload.png — XSS payload entered in search bar
- screenshots/02-xss-network-request.png — Network tab showing request was processed

POTENTIAL REAL-WORLD IMPACT

If the application ever renders search terms unsanitized in another context (email notifications, admin panels, logs), stored or reflected XSS could execute attacker-controlled JavaScript in victims' browsers, enabling session hijacking or credential theft.

PRACTICAL REMEDIATION

Sanitize all user input on the server side before processing. Reject or strip HTML tags from search parameters. Implement a Content Security Policy to reduce XSS impact.

4. OVERALL RISK SUMMARY

#	FINDING	SEVERITY	CATEGORY	DISCOVERY
1	SQL Injection — Admin Bypass	Critical	Injection	Manual
2	Sensitive Data Exposure via API	Medium	Information Disclosure	Manual / DevTools
3	XSS Input Accepted in Search	Medium	Cross-Site Scripting	Manual

Total findings: 3

Critical: 1

Medium: 2

Low: 0

5. EVIDENCE

screenshots/01-xss-search-payload.png	XSS payload in search bar
screenshots/02-xss-network-request.png	Network tab — search request
screenshots/03-api-sensitive-data-1.png	API response — sensitive fields (1)
screenshots/04-api-sensitive-data-2.png	API response — sensitive fields (2)
screenshots/05-auth-error-message.png	Auth-error message
screenshots/06-sqli-payload-entered.png	SQL injection payload entered
screenshots/07-sqli-admin-menu.png	Admin account menu after bypass
screenshots/08-sqli-admin-home.png	SQL injection payload entered
screenshots/12-sqli-admin-home.png	Admin session — home page